

A. CORE C HEADERS & FUNCTIONS

Since you're writing on paper, always remember to `#include` the correct headers at the top of your file.

1. <stdio.h> (Standard Input/Output)

This is for “printing” to the screen and “reading” from the keyboard.

- `int printf(const char *format, ...);`
 - Used for formatted output.
 - Returns the number of characters printed or a negative value on error.
 - **Example:** `printf("Age: %d, Name: %s\n", 25, "Bob");`
 - **Common Specifiers:**
 - `%d` or `%i`: Signed integer (int)
 - `%u`: Unsigned integer (unsigned int)
 - `%ld`: Long integer (long int)
 - `%zu`: `size_t` type (VERY common in C for sizes/indices)
 - `%f`: Floating point (double)
 - `%c`: Character (char)
 - `%s`: String (null-terminated char*)
 - `%p`: Pointer (prints address)
 - `%%`: Prints a literal '%' sign
- `int scanf(const char *format, ...);`
 - Used for formatted input.
 - Returns the number of items successfully read.
 - **CRITICAL C vs C++ GOTCHA:** `scanf` needs **pointers** to the variables it should fill. This is the #1 mistake.
 - **Example:**

```
c      int age;          char name[50];          printf("Enter
age and name: ");          int result = scanf("%d %49s", &age,
name); // <-- Notice '&age'          // 'name' is already a pointer (an
array decays to a pointer), so no '&'          // '%49s' is a security
measure to prevent buffer overflow
```

2. <stdlib.h> (Standard Library)

This is for memory management, string-to-number conversions, and program control.

- `void* malloc(size_t size);`
 - Allocates `size` bytes of **uninitialized** memory.
 - Returns a `void*` pointer to the new memory, or `NULL` on failure.
 - **You must always check for `NULL`.**
 - **You must `free()` this memory later.**

- **Example:** `int* arr = (int*)malloc(10 * sizeof(int));`
 - Note: In C, the cast `(int*)` is optional, but good practice. In C++, it's required.
- `void* calloc(size_t num, size_t size);`
 - Allocates memory for an array of `num` elements, each `size` bytes long.
 - **Difference from malloc:** Memory is **zero-initialized**.
 - **Example:** `int* arr = (int*)calloc(10, sizeof(int));` // All 10 ints are 0
- `void* realloc(void* ptr, size_t new_size);`
 - Resizes a memory block previously allocated by `malloc` or `calloc`.
 - `ptr` is the old pointer. `new_size` is the new total size in bytes.
 - Returns a pointer to the resized memory (which might be at a new location) or `NULL` on failure.
 - **If it returns a new pointer, the old ptr is automatically freed.**
 - **Common pattern:**

```
c      int* new_arr = (int*)realloc(arr, 20 *
sizeof(int));      if (new_arr == NULL) {          // Handle
error, 'arr' is still valid      free(arr);          } else {
arr = new_arr; // Update pointer      }
```
- `void free(void* ptr);`
 - Deallocates a block of memory previously allocated by `malloc`, `calloc`, or `realloc`.
 - Passing `NULL` to `free` is safe and does nothing.
 - **Calling free on the same pointer twice is undefined behavior (double free).**
- `int atoi(const char* str);`
 - Converts a string (`char*`) to an integer (`int`). (Stands for “ASCII to integer”).
- `void exit(int status);`
 - Terminates the program. `exit(0)` or `exit(EXIT_SUCCESS)` means success. `exit(EXIT_FAILURE)` means error.

3. <string.h> (String/Memory Functions)

This is for *raw memory manipulation*. These functions do not know about data types; they just move bytes.

- `void* memset(void* dest, int c, size_t count);`
 - Fills the first `count` bytes of the memory area pointed to by `dest` with the constant byte `c`.
 - **Use case:** Zeroing out a struct or array.
 - **Example:**

```
c      struct MyStruct s;      memset(&s, 0, sizeof(struct
MyStruct)); // Zeros the whole struct
```
- `void* memcpy(void* dest, const void* src, size_t count);`
 - Copies `count` bytes from `src` to `dest`.
 - **Important:** The memory areas **must not overlap**. Use `memmove` (also in `<string.h>`) if they might.
 - **Example:**

```
c      int src[] = {1, 2, 3};      int dest[3];
memcpy(dest, src, 3 * sizeof(int));
```
- `size_t strlen(const char* str);`
 - Returns the length of a string, **not** including the null-terminator (`\0`).
- `int strcmp(const char* s1, const char* s2);`

- Compares two strings.
- Returns:
 - `< 0` if `s1` is less than `s2`
 - `0` if `s1` is equal to `s2`
 - `> 0` if `s1` is greater than `s2`

4. Other Important Headers

- `<stddef.h>`:
 - Provides `size_t` (the type returned by `sizeof` and used for all memory function sizes).
 - Provides `NULL` (a null pointer constant).
- `<stdbool.h>`:
 - Provides `bool`, `true`, and `false`. (C++ has these built-in; C does not).
 - **Always include this** if you want to use `bool`. Otherwise, you must use `int` (0 for false, non-zero for true).
- `<stdint.h>`:
 - Provides fixed-width integer types like `int32_t`, `uint32_t`, `int64_t`. Very useful for algorithms to guarantee size.
- `<assert.h>`:
 - Provides `assert(expression)`. If the expression is false, the program aborts.
 - Excellent for checking pre-conditions and invariants (e.g., `assert(ptr != NULL)`).

B. C vs. C++: KEY DIFFERENCES & GOTCHAS

- **No new/delete:** You **must** use `malloc/free`.
- **Structs:** In C, you must write `struct MyStruct s;`. In C++, you can just write `MyStruct s;`
 - **C idiom:** Use `typedef` to mimic C++ behavior. ````c typedef struct Node { int data; struct Node* next; } Node;

Node* n = (Node*)malloc(sizeof(Node)); // Now you can just use 'Node' ````
- **No Pass-by-Reference (&):** C only has pass-by-value. To “pass by reference,” you must explicitly pass a pointer. ````c // C++ // void increment(int& x) { x++; } // int a = 5; increment(a);

// C equivalent void increment(int* x_ptr) { (*x_ptr)++; // Dereference the pointer to change the original value } int a = 5; increment(&a); // Pass the address ````
- **Strings:** C has no `std::string`. Strings are `char*` (pointers to a character) that are terminated by a null byte (`\0`). You are responsible for all memory management.
- **Boolean:** Use `<stdbool.h>` for `bool`.
- **No Function Overloading:** You cannot have two functions with the same name, even with different parameters.

C. COMPILER FLAGS EXPLAINED

You must write code that satisfies all these flags. `-Werror` means *any warning* will stop compilation.

- `-std=c17`: Use the C17 standard.
- `-pedantic`: Be extra strict about the standard.
- `-Wall, -Wextra`: Enable all standard and extra warnings. This is the source of most errors.
- `-Wformat-security`: Warns if `printf/scanf` format strings are mismatched with arguments.
- `-Wduplicated-cond`: Warns for `if (x > 5 && x > 5)`.
- `-Wfloat-equal`: Warns if you use `==` or `!=` with floats (due to precision issues).
- `-Wshadow`: Warns if a local variable has the same name as a global or outer-scope variable.
- `-Wconversion`: Warns about implicit conversions that might lose data (e.g., `int x = 5.5;`).
- `-Wjump-misses-init`: Warns if a `goto` or `switch` jumps over a variable's initialization.
- `-Wlogical-not-parentheses`: Warns for `!a && b`, which might be ambiguous. Prefers `(!a) && b`.
- `-Wnull-dereference`: Tries to warn if it detects a `NULL` pointer being used.
- `-Wvla`: Warns on Variable Length Arrays (e.g., `int n = 10; int arr[n];`).
 - **CRITICAL:** This flag plus `-Werror` **bans VLAs**. You **must** use `malloc` for dynamic-sized arrays.
- `-Werror`: **Treats all warnings as errors**. This is the key. Your code must be 100% warning-free.
- (Runtime/Debug Flags):
 - `-fstack-protector-strong`: Adds security checks to prevent stack buffer overflows.
 - `-fsanitize=undefined`: Turns on the Undefined Behavior Sanitizer (UBSan). Your code will crash if you do things like integer overflow, divide by zero, etc.
 - `-fno-sanitize-recover`: Tells UBSan to crash immediately, not try to continue.
 - `-g`: Include debug symbols.
 - `-fno-omit-frame-pointer`: Keep debug information (stack frames) for better debugging.

- -O1: Level 1 optimization.

D. PROBLEMATIC CODE PATTERNS (due to -Werror)

Any of these will likely cause a compilation error.

1. Using VLAs (Variable Length Arrays)

- **BAD:** `int n; scanf("%d", &n); int my_array[n];`
- **FIX:** `int n; scanf("%d", &n); int* my_array = (int*)malloc(n * sizeof(int));` (And check for NULL!)

2. Implicit Conversions (-Wconversion)

- **BAD:** `int x = 5.5;` (double to int, loses data)
- **BAD:** `char c = 1000;` (int to char, overflows)
- **FIX:** Be explicit. `int x = (int)5.5;`

3. Signed/Unsigned Mismatch (-Wall)

- `size_t` (from `sizeof`) is unsigned.
- **BAD:** `for (int i = 0; i < strlen(s); i++)`
- **FIX:** `for (size_t i = 0; i < strlen(s); i++)`
- **Very Bad:** `int x = -1; size_t y = 5; if (x < y) { ... }`
 - This is false! `x` is converted to a huge *unsigned* number.
- **FIX:** `if (x < (int)y) { ... }` (if you know `y` fits)

4. scanf without Pointers (-Wformat-security)

- **BAD:** `int x; scanf("%d", x);` (Will crash or worse)
- **FIX:** `int x; scanf("%d", &x);`

5. Unused Variables (-Wall)

- **BAD:** `int my_func() { int x = 5; return 0; }` (`x` is unused)
- **FIX:** Just remove it. If you *must* have it, cast to void: `(void)x;`

6. Comparing Floats (-Wfloat-equal)

- **BAD:** `double a = 0.1, b = 0.2; if (a + b == 0.3) { ... }` (This is false!)
- **FIX:** Check within a small epsilon. `c #include <math.h> // for fabs`
`double epsilon = 0.00001; if (fabs((a + b) - 0.3) < epsilon)`
`{ ... }`

7. Variable Shadowing (-Wshadow)

- **BAD:** `c int x = 10; for (int i = 0; i < 5; i++) { int x = i; // This 'x' shadows the outer 'x' printf("%d", x); }`
- **FIX:** Use different variable names.

8. Missing switch Cases (-Wall)

- If you switch on an enum, you must handle *every* enum value or have a default.
- **BAD:** `c enum State { A, B }; enum State s = A; switch (s) { case A: break; } // <-- Missing 'case B:' and 'default:'`
- **FIX:** `c switch (s) { case A: break; case B: break; default: // Or just handle both cases explicitly break; }`

9. Using Uninitialized Variables

- **BAD:** `int x; if (condition) { x = 10; } printf("%d", x);` (What if

- condition is false?)
- **FIX:** Always initialize variables. `int x = 0;`

Example code:

```
// So that the compiler doesn't complain about malloc(0).
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Walloc-size"
#define max(a, b) (a > b ? a : b)
#define min(a, b) (a < b ? a : b)

#include "zbior_ary.h"
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <stdio.h>

int64_t global_q;

int64_t get_remainder(int64_t a)
{
    // Get positive remainder mod q.
    return (a % global_q + global_q) % global_q;
}

SegmentVector construct_vector(int64_t remainder)
{
    SegmentVector result;
    result.allocated_size = 0;
    result.begin = malloc(0);
    result.end = result.begin;
    result.remainder = remainder;
    return result;
}

void push_back_segment(SegmentVector* vector, Segment* segment)
{
    uint64_t old_size = (uint64_t) (vector->end - vector->begin);

    if (old_size == vector->allocated_size)
    {
        uint64_t new_size = old_size * 2 + 1;
        vector->begin = realloc(vector->begin, new_size * sizeof(Segment));
        vector->end = vector->begin + old_size;
        vector->allocated_size = new_size;
    }

    *vector->end = *segment;
    vector->end++;
}

zbior_ary construct_zbior_ary()
{
    zbior_ary result;
    result.allocated_size = 0;
    result.begin = malloc(0);
    result.end = result.begin;
    return result;
}

void push_back_vector(zbior_ary* set, SegmentVector* vector)
{
    uint64_t old_size = (uint64_t) (set->end - set->begin);

    if (old_size == set->allocated_size)
    {
        uint64_t new_size = old_size * 2 + 1;
        set->begin = realloc(set->begin, new_size * sizeof(SegmentVector));
        set->end = set->begin + old_size;
        set->allocated_size = new_size;
    }

    *set->end = *vector;
    set->end++;
}

zbior_ary ciag_arytmetyczny(int int_a, int int_q, int int_b)
{
    int64_t a = int_a, b = int_b, q = int_q;
    global_q = q;
    int64_t remainder = get_remainder(a);
    Segment segment = {a, b + global_q};

    SegmentVector vector = construct_vector(remainder);
    push_back_segment(&vector, &segment);
}
```

```

    zbior_ary result = construct_zbior_ary();
    push_back_vector(&result, &vector);
    return result;
}

zbior_ary singleton(int a)
{
    return ciag_arytmetyczny(a, (int) global_q, a);
}

zbior_ary suma(zbior_ary A, zbior_ary B)
{
    zbior_ary result = construct_zbior_ary();
    SegmentVector* a_vector = A.begin;
    SegmentVector* b_vector = B.begin;

    // Iterate over A and B by the order of remainders, in order to iterate
    // over SegmentVectors with the same remainders simultaneously.
    while (a_vector < A.end || b_vector < B.end)
    {
        // If a SegmentVector in A doesn't have a corresponding SegmentVector
        // in B (as in holding Segments with the same remainder mod q), then we
        // can just copy it.
        if (b_vector == B.end ||
            (a_vector < A.end && a_vector->remainder < b_vector->remainder))
        {
            push_back_vector(&result, a_vector);
            a_vector++;
            continue;
        }
        if (a_vector == A.end ||
            (b_vector < B.end && a_vector->remainder > b_vector->remainder))
        {
            push_back_vector(&result, b_vector);
            b_vector++;
            continue;
        }
    }

    Segment* a_segment = a_vector->begin;
    Segment* b_segment = b_vector->begin;

    SegmentVector result_vector = construct_vector(a_vector->remainder);

    // Iterate over the Segments of A and B simultaneously by their begin
    // in order to handle overlapping segments. current_segment holds a
    // sum of the current group of overlapping segments. Once the next
    // segment in A or B doesn't overlap with current_segment,
    // current_segment is added to the result.
    while (a_segment < a_vector->end || b_segment < b_vector->end)
    {
        Segment current_segment;
        bool first_segment = true;

        while (a_segment < a_vector->end || b_segment < b_vector->end)
        {
            // The first segment in the group has to be handled
            // differently, because current_segment isn't initialized yet.
            // The segment with a smallest begin out of
            // {a_segment, b_segment} is chosen and pointed to by
            // new_segment.
            bool a_not_end = a_segment < a_vector->end;
            int64_t a_begin = a_not_end ? a_segment->begin : LLONG_MAX;

            bool b_not_end = b_segment < b_vector->end;
            int64_t b_begin = b_not_end ? b_segment->begin : LLONG_MAX;

            Segment** new_segment = a_begin < b_begin ? &a_segment : &b_segment;

            if (first_segment)
            {
                // If new_segment is the first Segment in the group, we
                // just set current_segment to new_segment.
                current_segment = **new_segment;
                first_segment = false;
                (*new_segment)++;
            }
            else
            {
                // If current_segment and new_segment don't touch, we leave
                // new_segment for later. If they do, we extend
                // current_segment to the end of new_segment and increment
                // the pointer to which new_segment points to.
                if (current_segment.end < (*new_segment)->begin) break;
                int64_t new_end = (*new_segment)->end;
                // We pick new_segment by the order of begin, so it's not
                // guaranteed that they are also ordered by end, so a
                // particular new_segment may not extend current_segment.
                current_segment.end = max(current_segment.end, new_end);
                (*new_segment)++;
            }
        }
    }
}

```

```

        push_back_segment(&result_vector, &current_segment);
    }

    push_back_vector(&result, &result_vector);
    a_vector++;
    b_vector++;
}

return result;
}

zbior_ary iloczyn(zbior_ary A, zbior_ary B)
{
    zbior_ary result = construct_zbior_ary();
    SegmentVector* a_vector = A.begin;
    SegmentVector* b_vector = B.begin;

    // Iterate over A and B by the order of remainders, in order to iterate
    // over SegmentVectors with the same remainders simultaneously.
    while (a_vector < A.end && b_vector < B.end)
    {
        int64_t a_remainder = a_vector->remainder;
        int64_t b_remainder = b_vector->remainder;

        // If a SegmentVector doesn't have a corresponding SegmentVector in the
        // other set, then we can skip it.
        if (a_remainder < b_remainder)
        {
            a_vector++;
            continue;
        }
        if (a_remainder > b_remainder)
        {
            b_vector++;
            continue;
        }

        Segment* a_segment = a_vector->begin;
        Segment* b_segment = b_vector->begin;

        SegmentVector result_vector = construct_vector(a_remainder);

        // We iterate over the SegmentVector's segments simultaneously in
        // order to handle overlapping segments in the same iteration.
        while (a_segment < a_vector->end && b_segment < b_vector->end)
        {
            // new_segment is set to the intersection of a_segment and
            // b_segment and added to result_vector if not empty. Notice that
            // since segments in a_vector and b_vector don't touch, then the
            // intersections also don't touch, so we don't have to handle that
            // case. One segment from a_vector may overlap with two segments
            // from b_vector, so we have to increment the segment with smaller
            // end to make sure such cases are handled.
            Segment new_segment;
            new_segment.begin = max(a_segment->begin, b_segment->begin);
            new_segment.end = min(a_segment->end, b_segment->end);

            if (new_segment.begin < new_segment.end)
            {
                push_back_segment(&result_vector, &new_segment);
            }

            if (a_segment->end < b_segment->end) a_segment++;
            else b_segment++;
        }

        // We want to make sure that zbior_ary doesn't hold empty
        // SegmentVectors.
        int64_t result_vector_size = result_vector.end - result_vector.begin;
        if (result_vector_size > 0) push_back_vector(&result, &result_vector);

        a_vector++;
        b_vector++;
    }

    return result;
}

zbior_ary roznica(zbior_ary A, zbior_ary B)
{
    zbior_ary result = construct_zbior_ary();
    SegmentVector* a_vector = A.begin;
    SegmentVector* b_vector = B.begin;

    while (a_vector < A.end)
    {
        int64_t a_remainder = a_vector->remainder;

        // If there is no corresponding SegmentVector in B, then we can just
        // add a_vector. If there is no a_vector with the same remainder as

```



```

// b_vector, then this b_vector can be skipped.
if (b_vector == B.end || a_remainder < b_vector->remainder)
{
    push_back_vector(&result, a_vector);
    a_vector++;
    continue;
}
if (a_remainder > b_vector->remainder)
{
    b_vector++;
    continue;
}

Segment* a_segment = a_vector->begin;
Segment* b_segment = b_vector->begin;

SegmentVector result_vector = construct_vector(a_remainder);

for (; a_segment < a_vector->end; a_segment++)
{
    // For each a_segment, we iterate over all b_segment that
    // overlap with it and add the non-intersection parts to
    // result_vector. new_begin is used to store the begin of the
    // next part of a_segment.
    int64_t new_begin = a_segment->begin;

    for (; b_segment < b_vector->end; b_segment++)
    {
        // If all of a_segment is used up, we can break the loop
        // instead of dealing with degenerate cases.
        if (new_begin >= a_segment->end) break;

        // If b_segment starts after new_begin, it divides what
        // currently remains of a_segment into two parts.
        if (new_begin < b_segment->begin)
        {
            // This may be the last part of a_segment.
            int64_t new_end = min(a_segment->end, b_segment->begin);
            Segment new_segment = {new_begin, new_end};
            push_back_segment(&result_vector, &new_segment);
        }

        // It's possible for there to be b_segment before this
        // a_segment, so the begin of the next part is the max of
        // new_begin and the end of b_segment.
        new_begin = max(new_begin, b_segment->end);

        // Before incrementing b_segment, we have to check whether it
        // ends after a_segment, because if it does, then it may
        // overlap with the next a_segment.
        if (b_segment->end > a_segment->end) break;
    }

    // The last part of a_segment may not have a b_segment after it, so
    // it has to be added separately.
    if (new_begin < a_segment->end)
    {
        Segment new_segment = {new_begin, a_segment->end};
        push_back_segment(&result_vector, &new_segment);
    }
}

int64_t result_vector_size = result_vector.end - result_vector.begin;
if (result_vector_size > 0) push_back_vector(&result, &result_vector);

a_vector++;
b_vector++;
}

return result;
}

bool nalezy(zbior_ary A, int int_b)
{
    // Do a double binary search, first over the remainders, and then over the
    // Segments. begg is set to the remainder/Segment which contains b.
    int64_t b = int_b;
    if (A.begin == A.end) return false;

    int64_t remainder = get_remainder(b);
    int64_t begg = 0;
    int64_t endd = A.end - A.begin;

    while (endd - begg > 1)
    {
        int64_t midd = (begg + endd) / 2;
        if (A.begin[midd].remainder <= remainder) begg = midd;
        else endd = midd;
    }

    if (A.begin[begg].remainder != remainder) return false;
}

```

```

SegmentVector vector = A.begin[begg];
begg = 0;
endd = vector.end - vector.begin;

while (endd - begg > 1)
{
    int64_t midd = (begg + endd) / 2;
    if (vector.begin[midd].begin <= b) begg = midd;
    else endd = midd;
}

Segment closest_segment = vector.begin[begg];
if (closest_segment.begin <= b && b < closest_segment.end) return true;
else return false;
}

unsigned moc(zbior_ary A)
{
    uint64_t result = 0;

    for (SegmentVector* vector = A.begin; vector < A.end; vector++)
    {
        for (Segment* segment = vector->begin; segment < vector->end; segment++)
        {
            int64_t element_count = (segment->end - segment->begin) / global_q;
            result += (uint64_t) element_count;
        }
    }

    return (unsigned) result;
}

unsigned ary(zbior_ary A)
{
    uint64_t result = 0;

    for (SegmentVector* vector = A.begin; vector < A.end; vector++)
    {
        result += (uint64_t) (vector->end - vector->begin);
    }

    return (unsigned) result;
}

// comp and print are only used for tests and are not part of the zbior_ary
// functionality themselves. They may be ignored by reviewers and graders.
int comp(const void* a, const void* b)
{
    const int64_t* int_a = (const int64_t*) a;
    const int64_t* int_b = (const int64_t*) b;
    return (int) (*int_a - *int_b);
}

void print(zbior_ary A)
{
    int64_t* elements = malloc(moc(A) * sizeof(int64_t));
    uint64_t element_count = 0;

    for (SegmentVector* vector = A.begin; vector < A.end; vector++)
    {
        for (Segment* segment = vector->begin; segment < vector->end; segment++)
        {
            for (int64_t element = segment->begin; element < segment->end; element += global_q)
            {
                elements[element_count] = element;
                element_count++;
            }
        }
    }

    qsort(elements, element_count, sizeof(int64_t), comp);

    for (uint64_t i = 0; i < element_count; i++) printf("%ld ", elements[i]);
    printf("\n");
}

```